

Sistemas Operativos
3 de febrero de 2021, Curso 2020-21
Facultad de Informática, UCM

Cuestión 1. El sistema de ficheros de un computador sigue un esquema basado en FAT-16 (utilizando por tanto direcciones de bloque con tamaño de 2 bytes). Además, se sabe que el tamaño de un directorio siempre es un múltiplo exacto del tamaño de bloque. En un determinado momento, la tabla FAT contiene la siguiente información:

0	1	2	3	4	5	6	7	8	9	10	11	12	13
EOF	EOF	10	30	44	13	34	21	22	23	1	32	40	20
14	15	16	17	18	19	20	21	22	23	24	25	26	27
28	27	6	33	26	11	35	38	36	3	50	5	52	48
28	29	30	31	32	33	34	35	36	37	38	39	40	41
45	16	EOF	EOF	EOF	9	12	42	47	4	31	19	EOF	55
42	43	44	45	46	47	48	49	50	51	52	53	54	55
51	25	53	37	43	18	24	46	41	8	39	EOF	2	29

En este sistema, la ruta de un fichero es /SO/Exámenes/Febrero2021.docx. De los directorios y el fichero se conocen los siguientes datos:

	Primer bloque	Tamaño	Tipo
SO	17	384 bytes	D
Exámenes	7	256 bytes	D
Febrero2021.docx	54	200 bytes	F

- a) Represente esquemáticamente la misma situación de disco, pero suponiendo que, en vez de FAT, se dispone de un sistema de ficheros tipo UNIX. En este sistema, la estructura del inodo está compuesta por tres punteros directos, un puntero indirecto simple y un puntero indirecto doble.
- b) Indique y explique el número máximo y mínimo de accesos a bloques de disco que se requieren para leer todo el fichero Febrero2021.docx utilizando:
- a) El sistema de asignación basado en FAT descrito inicialmente. Supóngase que el primer bloque del directorio SO es conocido, y no hay ningún bloque de la FAT cargado inicialmente en memoria.
- a) El sistema de asignación tipo UNIX indicado. Supóngase que el inodo de SO se encuentra cargado en memoria y que cada inodo ocupa exactamente un bloque de disco.

Cuestión 2. Se desea simular con hilos un pequeño torneo de poker entre amigos. La función de entrada de los hilos que simulan los jugadores tiene la siguiente estructura:

```
void play(void);

void *player(void *arg)
{
    while (1) {
        seat_at_table();
        play();
        leave_table();
    }
}
```

La función `seat_at_table` debe garantizar que cada jugador accede a la mesa en estricto orden de llegada, mientras no haya empezado la siguiente partida. Asimismo, el hilo no debe ejecutar la función `play` mientras no se hayan sentado a la mesa los 4 jugadores que jugarán la siguiente partida. Los hilos de los 4 jugadores que juegan juntos deben poder ejecutar la función `play` concurrentemente.

Cuando la partida termine (retorno de la función `play`) el último jugador en dejar la mesa (`leave_table`) deberá indicar al resto que la partida actual ha finalizado y que puede empezar una nueva partida.

Implementar las funciones `seat_at_table` y `leave_table` para que los hilos se sincronicen correctamente, añadiendo las variables globales necesarias.

Solución:

```
#define N 18
#define M 4

mutex_t mtx;
cond_t tfree, tfull;
int round_started = 0;
int table_count = 0;
int turn = 0;
int ticket = 0;

void seat_at_table(void)
{
    int myticket;

    mutex_lock(&mtx);
    myticket = ticket++;
    while (round_started || myticket != turn)
        cond_wait(&tfree, &mtx);

    turn++;
    table_count++;
    if (table_count == 4) {
        cond_broadcast(&tfull);
        round_started = 1;
    } else {
        cond_broadcast(&tfree);
        while (table_count < 4)
            cond_wait(&tfull, &mtx);
    }
    mutex_unlock(&mtx);
}

void leave_table(void)
{
    mutex_lock(&mtx);
    table_count--;
    if (table_count == 0) {
        round_started = 0;
        printf("Round finished, table is free\n");
        cond_broadcast(&tfree);
    }
    mutex_unlock(&mtx);
}
```

Cuestión 3. Un SO aplica una política de planificación con colas multinivel (3 niveles) y realimentación a las tareas siguiendo las siguientes directrices:

- Las tareas nuevas entran al nivel 1.
- En todos los niveles se aplica planificación **Round-Robin**, con quantos crecientes: $q=2$ (nivel1), $q=3$ (nivel2), $q=4$ (nivel3).
- Las tareas del nivel 1 que durante una ráfaga de CPU agoten su cuanto y necesiten más CPU pasarán al nivel 2. Si la tarea finaliza su ráfaga justo al agotar su cuanto permanece en el nivel.
- Las tareas del nivel 2 que durante una ráfaga de CPU agoten su cuanto y necesiten más CPU pasarán al nivel 3. Si la tarea finaliza su ráfaga justo al agotar su cuanto permanece en el nivel.
- Cada 15 unidades de tiempo, comenzando en el instante 0, (instantes 15,30,...) todos los procesos que se encuentren en el tercer nivel, o estén bloqueados por entrada salida pero fueron planificados por última vez desde este nivel, recibirán una actualización de su prioridad y pasarán a la cola de nivel 1. Esta mejora es independiente del instante del instante de llega a dicho nivel. Si hay varias tareas que mejoran su prioridad, el orden que tuvieran entre ellas en el nivel 3 se conserva en el nivel 1.
- Las tareas que retornen de entrada salida se mantendrán en el nivel en el que fueron planificadas por última vez, independientemente del nivel.

A) Identificas problemas que puede presentar este algoritmo de planificación?. En caso afirmativo, describirlos razonando la respuesta.

B) Aplíquelo a las siguientes trazas:

Tarea	Llegada	CPU	E/S	CPU	E/S	CPU
T1	0	1	3	1	4	3
T2	2	7	1			
T3	1	3	1	1	1	1
T4	3	5	1	1		

Completar el diagrama temporal, indicando también los procesos preparados que hay en cada una de las colas. Al rellenar el diagrama temporal, se ha de usar el siguiente convenio:

- En ejecución: marcar con X
- Bloqueado por E/S: marcar con O
- Listo para ejecutar: marcar con –

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
T1																								
T2																								
T3																								
T4																								
Q1																								
Q2																								
Q3																								

Cuestión 4. Un sistema utiliza gestión de memoria virtual con paginación bajo demanda con páginas de 2KiB, direcciones de memoria de 32 bits y memoria principal de 8GB. En dicho sistema se desea ejecutar el siguiente programa:

```
#include <stdio.h>
/* resto de includes */
...

#define N 2048
int A[N] = {0};
int *B;

void* copyv(void* arg);

int main(int argc, char* argv[])
{
    int pid, i;
    pthread_t th1, th2;
    int arg1[2], arg2[2];

    // Punto A
    for (i=0; i<N; i++)
        A[i] = i+1;
    // Punto B
    printf("Initialization was OK");
    // Punto C
    B=malloc(N*sizeof(int))
    // Punto D
    arg1[0] = 0;    arg1[1] = N/2 - 1;
    arg2[0] = N/2; arg2[1] = N-1;
```

```
    pthread_create(&th1, NULL,
        copyv, (void*) arg1);
    pthread_create(&th2, NULL,
        copyv, (void*) arg2);
    // Punto E
    pthread_join(th1, NULL);
    pthread_join(th2, NULL);
    // Punto F
    for (i=0; i<N; i++)
        printf("B[%i]=%i\n", i, B[i]);
    // Punto G
    return 0;
}

void* copyv(void* arg)
{
    int* params = (int*) arg;
    int i;
    for (i=params[0]; i <= params[1]; i++)
        B[i] = A[i];

    return NULL;
}
```

Considerando que:

- Las llamadas al sistema y el resto de funciones de librería que se incluyen en el programa nunca fallan.
- Hay suficientes marcos de página libres en el sistema.
- Todas las funciones externas se incluyen en una única biblioteca, donde no se definen variables globales. Dicha biblioteca se enlazó de forma estática al crear el ejecutable.
- La única sección de texto (código) presente en el ejecutable tras la compilación y enlazado estático del programa ocupa 103KiB.
- El contenido inicial de la pila al lanzar a ejecución el programa ocupa 128 bytes.
- Un entero en C ocupa 4 bytes.

Responda a las siguientes preguntas:

- a. Identifique qué regiones lógicas (y su tamaño en páginas) constituyen la imagen de memoria del proceso multihilo en los puntos marcados en el código como A, B, C, D, E, F y G.
- b. ¿Cuántos fallos de página se producirán al pasar del punto A al punto B?. ¿Y del punto F al G? Justifique sus respuestas.

Cuestión 5. Un proceso en UNIX ejecuta el siguiente código:

```
1  #include <fcntl.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <sys/types.h>
5  #include <unistd.h>
6  #include <sys/wait.h>
7  #include <pthread.h>
8
9  #define N 5
10
11 int *fibseq;
12 char message[] = "Bye Fibonacci";
13 pthread_t threads[N];
14
15 void * fibo (void * arg ) {
16
17     int i= (int) arg;
18
19     if(i==0)
20         fibseq[i] = 0;
21     else if(i==1)
22         fibseq[i] = 1;
23     else
24         fibseq[i] = fibseq[i - 1] + fibseq[i - 2];
25
26     printf ("fib %d: %d\n",i,fibseq[i]);
27     pthread_exit (0);
28     return NULL;
29 }
30
31
32 int main(){
33
34     int i, j;
35
36     fibseq = (int *) malloc(N * sizeof(int));
37
38     pid_t pid = fork();
39
40     for(i=0; i<N; i++)
41         fibseq[i] = -1;
42
43     if (pid == 0){
44         for(i=0; i<N; i++)
45             pthread_create (&threads[i],
46                             NULL, fibo, (void *)i);
47
48         for (j=0; j<N; j++)
49             pthread_join ( threads[j], NULL);
50
51         exit (0);
52     }
53     else {
54         execlp("/bin/echo", "/bin/echo", message, NULL);
55     }
56
57     printf ("pid= %d ppid= %d value %d= %d\n",
58            getpid(), getppid(), N, fibseq[N-1]);
59
60     return 0;
61 }
```

Asumiendo que el proceso original tiene PID 100 (y es hijo de *Init*), se pide:

- Explica razonadamente cuántos procesos/hilos serán creados y cuantos ejecutarán potencialmente de forma concurrente.
- Justifica si se puede determinar o no el orden en que se ejecutarán estos procesos/hilos, indicando en tal caso dicho orden. En cualquier caso, indica una posible salida del código anterior.
- Indica una posible salida si cambiamos la función `execlp("...")` por `wait(NULL)`.

Con la función `execlp` el proceso padre cambia el programa en ejecución por el programa `echo message`, mostrando por pantalla el mensaje de "Bye Fibonacci" (podrá aparecer en cualquier orden y no necesariamente al final) y por lo tanto, no terminará el programa inicial. Si lo cambiamos por `wait(NULL)` el proceso padre esperará por la terminación del proceso hijo y mostrará por pantalla el último `printf`: `pid=100 ppid=1 value5=-1`